

Data Engineering Optimizations for Secure V2X Communications

Capstone Project

Final Report

Fall 2025

Team: Seonuk Kim, Harsh Bhaskar, Stephen Guzzaralapudi

Mentors: Dr. Lanier Watkins, Dr. Ahmed Abdo (APL), Ilya Sabnani

Abstract

This project develops a lightweight data platform to improve V2X security and resilience in vehicular fog computing (VFC) environments, and we demonstrate that the proposed platform is optimized for fog computing settings. We also demonstrate a novel approach to V2X cybersecurity by applying autonomous cybersecurity principles consisting of micro-intrusion detection systems (IDSs) and an aggregator for better explainability and autonomy. To our knowledge, this approach has not been applied to V2X settings before. Finally, this project applies the autonomous cybersecurity approach using our proposed data platform to detect and filter out malicious Basic Safety Messages (BSMs) during a protocol-compliant BSM flooding and fabricated data attack.

1. Introduction

1.1: Significance

The past decade has seen significant progress towards level 5 (L5) autonomous driving, defined by the Society of Automotive Engineers (SAE) as vehicles having full control over both driving and navigation. For example, Waymo's taxi service has demonstrated SAE Level 4 autonomy within limited geofenced areas, with the vehicle assuming all responsibility in driving and navigation [1]. One of the key technologies enabling these advancements is **vehicle-to-everything (V2X)** communications to improve the vehicles' ability to perceive their surroundings, plan their movements, and control their decisions [2]. As a result, finding methods to optimize V2X communications on the data engineering side, as well as securing the overall process, has garnered considerable academic and industry attention.

1.2: Problem Definition

Real-life deployments of V2X communications for capabilities such as local dynamic maps (LDMs), vehicle platooning, and cooperative perception have been increasing but are still relatively limited. As a result, a number of simulators have been developed, including the **Vehicular Secure Network Open Simulator (VESNOS)** [3]; however, based on our research, there are no simulators that integrate in a data platform from which the voluminous amounts of data generated from simulations are stored and processed for future use, such as training and/or applying algorithms to detect anomalies stemming from simulated cyberattacks.

Additionally, today's growing reliance on vehicular cloud computing (VCC) and vehicular edge computing (VEC) can be considered a potential threat. Loss in functionality is possible through forces of nature, service provider maintenance, or intentional malicious actions disrupting the links between V2X edge nodes and the edge servers or central data centers. This becomes particularly problematic when cyberattack detection and mitigation capabilities are hosted on these data centers, leading to a loss in availability of these services when the links are down. For example, the recent AWS and Cloudflare outages demonstrate the risk of complete reliance on cloud computing [4], especially in an environment that is highly sensitive to latency and availability in general.

1.3: Capstone Objectives

To bolster the resilience aspect of today's V2X infrastructure, which heavily relies on vehicular edge computing (VEC) and vehicular cloud computing (VCC), this project seeks

to develop a lightweight data platform that can be used in **vehicular fog computing (VFC)** settings¹. A visualization of these three concepts is shown below:

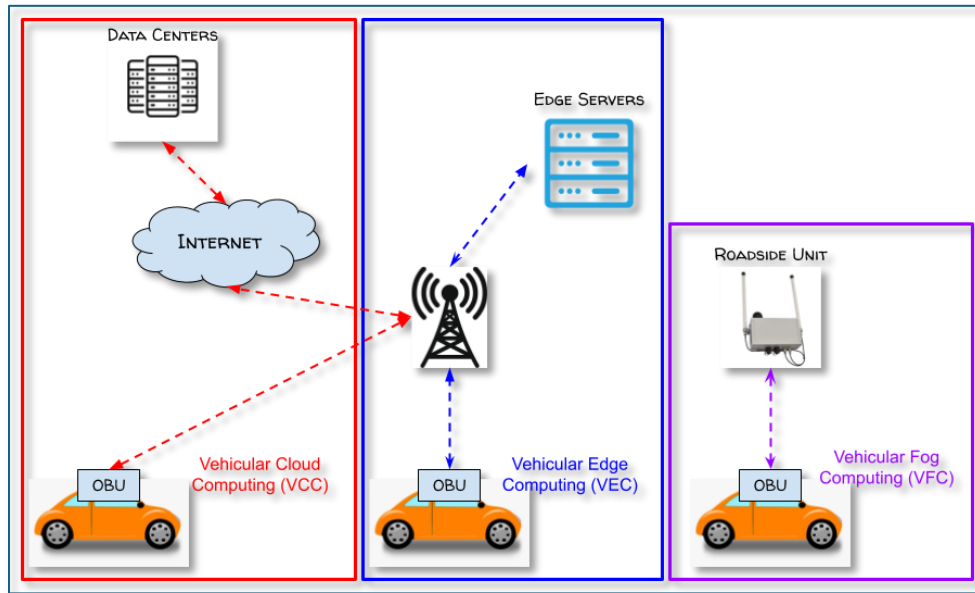


Figure 1: From left-to-right, summary of: [left] static vehicular cloud computing (VCC); [center] vehicular edge computing (VEC), and [right] vehicular fog computing (VFC).

Most importantly, vehicular fog computing extends vehicular cloud computing to the edge, and for the scope of this project, the data platform is thought to be operating on roadside units (RSUs) as well as other V2X infrastructure pieces that lay **at most one hop away** from connected vehicles. This means that the reliance on edge servers and/or data centers is eliminated at the cost of limited computing resources, helping address potential situations when links between the V2X edge nodes and the data centers/edge servers are down.

In addition, another challenge that we hope to address is anomaly detection in limited data availability settings to detect attacks against V2X communications, since vehicular fog computing is a decentralized approach to computing. We use our proposed data platform to apply an autonomous cybersecurity system consisting of micro-intrusion detection systems (IDSs) for different aspects of V2X traffic, as well as an aggregator that combines the outputs of these micro-IDSs to determine whether the observed traffic is malicious. For the scope of this project, the system will be applied to detect and mitigate a flooding denial-of-service attack, where an adversary attempts to overwhelm resources by flooding the target with protocol-compliant Basic Safety Messages (BSMs).

¹ Definitions of “vehicular cloud computing” vary, with some referring to VCC as shared computing by creating a *local* cloud (dynamic VCC), whereas others as the traditional cloud computing via a centralized data center (static VCC). We refer to the latter when we refer to VCC in this context.

To summarize, our capstone project aims to:

1. Develop a lightweight, efficient data platform with the full functionality of message brokering/queuing, data storage, and data processing that can be integrated into V2X simulators.
2. Apply this data platform to train and deploy machine learning models making up an autonomous cybersecurity system, then test the effectiveness of such system against a BSM-flood denial-of-service attack.

2. Literature Review: Background on V2X Communications

This section aims to present background information necessary to understand V2X. First, we define V2X communications, including its various components and its underlying technologies; understanding the radio access technologies, for example, provides a better idea of the demands the data platform must handle. We then introduce SUMO, the simulator in which our data platform will be integrated. We also provide a literature review on data engineering in V2X settings, as well as vehicular cloud computing (VCC) and fog computing (VFC).

2.1: Introduction to Vehicle-to-Everything (V2X) Communications

Vehicle-to-Everything, or V2X, communications refer to transmissions of data between vehicles and any other entity, such as roadside infrastructure, pedestrians, and other vehicles. Onboard units (OBUs) on connected vehicles (CVs) transmit data to/from these other entities, sending the vehicle's current position, velocity, acceleration, and more. When a vehicle exchanges data with other vehicles, this is known as Vehicle-to-Vehicle (V2V) communications. V2V can be applied for safety services such as providing forward collision warnings, as well as non-safety services such as cooperative adaptive cruise control (CACC), which improves traffic efficiency [5].

Vehicle-to-Infrastructure, or V2I, on the other hand, refers to communications between the vehicle and roadside infrastructure, such as roadside units (RSUs) and application servers. These RSUs provide important information for CVs, such as the status of road traffic and weather conditions, allowing vehicles to respond accordingly – for example, they can adjust for sharp curves in their current routes and reduce their speed for upcoming road segments containing hazards [6]. RSUs often serve as the bridge between the CVs and the data centers that collect mammoth amounts of data to create Intelligent Transportation Systems (ITS) [7].

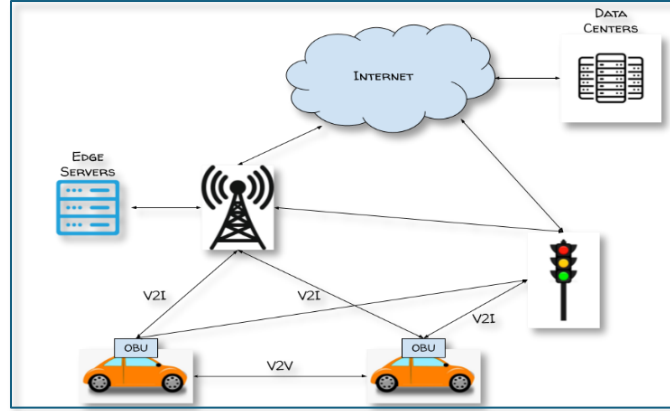


Figure 2: Summary of V2X, demonstrating vehicle-to-vehicle (V2V) and vehicle-to-infrastructure (V2I) communications. Data sent through V2X can be processed at edge servers or at centralized data centers.

The radio access technology (RAT) used for V2X communications plays a significant role. Dedicated Short-Range Communications (DSRC), which uses Wi-Fi (802.11p), can enable immediate data exchange between CVs and RSUs without any authentication, but, as the name suggests, any communications are limited to short ranges and low capacities [8]. DSRC as a protocol suite is used in Wireless Access in Vehicular Environment (WAVE) and allows for V2V and V2I communications in a Vehicular Ad-Hoc Network (VANET).

Cellular V2X can also be used for V2X communications, using either 3G, LTE, or 5G New Radio (NR-V2X). By handling much higher demands in bandwidth, latency, and throughput, C-V2X can accommodate the high network loads required from V2X data, which becomes a critical need as estimates back from 2020 suggested a CV would transmit/consume terabytes of V2X data per day [9]. Although DSRC has been found to perform better than LTE C-V2X in advanced V2X applications such as autonomous driving, 5G NR C-V2X is expected to provide superior performance than DSRC, not to mention serve as a stronger future-proof RAT [10].

2.2: V2X Messages

The SAE J2735 standard defines the different messages sent in V2X communications [11]. In this section, we highlight the specifications for Basic Safety Messages (BSMs), which will be applied in our machine learning models to detect and mitigate BSM flooding attacks. For a summary on the data usage and throughput rates for each type of message, refer to the appendix. Because Signal Phase and Timing (SPaT) messages have higher maximum requirements on the road, this section also explains the functionality of SPaT messages to provide background information needed to test and develop the data platform.

V2X Phase	Message Types	Urban	Suburban	Rural/Highway
Awareness Driving	Basic Safety Message (BSM)	1140	2280	3800

	Signal Phase and Time (SPaT) MAP Message (MAP) In-Vehicle-Information Message (IVI)	12000	30000	60000
	Personal Safety Message (PSM)	350	350	3500
Sensing Driving	Collective Perceptive Message (CPM)	3000	6000	10000
Cooperative Driving	Manuever Coordination Message (MCM)	3000 20000	6000 20000	10000 20000
	Platooning Control Message (PCM)			
TOTAL (Hz)		39490	64630	107300

Figure 3: Expected total bytes per second usage for each message type, based on the standard proposed by [12].

2.2.1: Basic Safety Messages (BSMs)

Basic Safety Messages, or BSMs, are used to exchange data on a vehicle's state for safety purposes, such as Traffic Jam Warnings, Emergency Vehicle Warnings, and Intersection Collision Warnings. The SAE J2735 requires BSMs to consist of fields such as the speed, latitude/longitude, elevation, and steering wheel angle of the vehicle, as well as additional extensions as needed for different vehicles and applications. According to the Car-2-Car Communication Consortium (C2C-CC), each BSM is expected to be 380 bytes, with anywhere between 3 and 10 BSMs sent every second based on whether the setting is urban, suburban, or rural [12].

Description	Field Names	
Provides information on the vehicle's state for safety purposes and applications.	MsgCount	TemporaryId
	Dsecond	Latitude
	Longitude	Elevation
	PositionalAccuracy	TransmissionState
	AccelerationSet4Way	Heading
	SteeringWheelAngle	VehicleSize
	BrakeSystemStatus	Speed

Figure 4: Properties of Basic Safety Messages (BSM).

2.2.2: Signal Phase and Time (SPaT) Messages

Signal Phase and Time, or SPaT, messages contain the status of a signal at an intersection, helping vehicles identify timing for when the next phase of the signal is expected to occur. Traffic signal controllers or RSUs broadcast SPaT messages, playing a crucial role in Infrastructure-to-Vehicle (I2V) communications. These are typically combined with MAP messages that contain information on the geometric layout of an intersection. C2C-CC expects SPaT and MAP messages to be transmitted 10 times every second, with each message coming to around 1200 bytes.

2.3: Vehicular Cloud Computing (VCC) and Vehicular Fog Computing (VFC)

There are two types of vehicular cloud computing (VCC) described in literature – static VCC and dynamic VCC. This section briefly describes the two concepts, as well as the challenges that each must address. Vehicular fog computing is explained next, including how it addresses some of the challenges VCC faces.

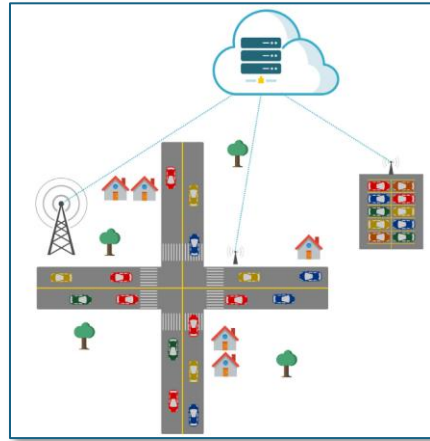


Figure 5: Illustration from [13] illustrating both types of static vehicular cloud computing.

2.3.1: Static Vehicular Cloudlets

Static vehicular cloudlets refer to situations when data is transmitted to and processed at a centralized location that is stationary. It includes the traditional cloud computing architectures, where edge nodes interact with centralized data centers for cloud services, such as AWS and Azure. When applied to V2X environments, this introduces a significant problem – availability. Safety applications running on the in-transit OBUs cannot reliably connect to the cloud servers with the low latency required by US standards. According to the National Highway Traffic Safety Administration’s October 2016 report, the maximum latency for LTE-based vehicle-to-network applications is 1000 milliseconds [13]. A vehicle traveling 60 miles per hour on a highway would travel 88 feet in 1000 milliseconds, and although this satisfies the DSRC V2X communication range of around 1,000 feet [8], the conditions on the road may change dramatically within this distance and time.

There is another availability problem with applying the traditional cloud computing architecture to V2X settings. If the links from the infrastructure units to the Internet are disrupted, any safety applications running on either the RSUs or OBUs will not be able to receive important safety information at all. Safety information is not limited to data on nearby CVs in the form of BSMs, but also security-related data as well. For example, because intensive and comprehensive intrusion detection systems cannot be run on the limited resources available on OBUs and RSUs, they may be offloaded to compute-heavy

data centers. Any outage affecting this security-related data relay will be detrimental for normal V2X communication functions.

Another variant of static vehicular cloudlets is the cloud computing architecture customized for V2X environments. This includes nearby vehicles that are stationary as well, but as an aggregate can function as local “clouds” providing resources for CVs in need. In this scenario, unused vehicles share their underutilized resources, whether this be in storage, computation, networking, or applications [14]. For example, [15] proposes using CVs in a long-term parking lot at an international airport as a “data center” to share available resources.

Although this type of static vehicular cloudlets reduces latency significantly by moving the “data center” closer to the CVs requiring the resources, there remain complex problems to be solved. Privacy is one of the biggest problems that has not been completely addressed yet, as there is still no mechanism to establish trust in such shared pools [16]. It is possible for an adversary to join as a member of the local static vehicular cloudlet, with the possibility of injecting false information into important safety applications. The willingness of CV owners to enable shared computing is another question that has not been focused on as well – there is a lack of research focusing on this issue, raising doubt on the viability of implementing this in the real world.

2.3.2: Dynamic Vehicular Cloudlets

In addition to static vehicular cloudlets, a newer form of VCC has been proposed, with dynamic vehicular cloudlets handling volatile traffic conditions. This has become an active area of research, as communication protocols in this type of VCC must accommodate the highly unstable topology of communication networks [17]. CVs in transit may only be able to share their resources for a brief period of time, in which the cloud member must authorize, authenticate, and share resources with the requesting CV. Studies such as [18] have proposed different dynamic resource allocation and scheduling algorithms to meet the low-latency requirements of V2X safety applications. Even if a reliable and resilient technical solution is developed, the same problem as the second type of static vehicular cloudlets is still present – there is no solution yet to guarantee the privacy and trust of the vehicular cloudlet [16].

2.3.3: Vehicular Fog Computing (VFC)

Vehicular fog computing (VFC) has been proposed as a solution to the potential latency issues arising from vehicular cloud computing. Similar to traditional fog computing, VFC takes advantage of idle resources unutilized by nodes between the vehicles and the clouds – for example, the unused resources on RSUs. In other words, it is not reliant on cloud-based Internet-of-Vehicles communications like VCC, instead depending on the

distributed deployment of different applications towards the edge, between the end device and the cloud [19]. It is important to note that some papers have different definitions of “vehicular fog computing,” defining it as dynamic vehicular cloud computing where a “fog” of unused resources around the application is utilized. This is not the type of VFC this capstone project analyzes – we instead refer to VFC as computing done on RSUs and other infrastructure pieces sitting between the applications and the cloud.

This is highly relevant, as not only does this type of VFC decrease the latency for applications using fog computing, but it also increases the scalability of these applications. Decision-making is moved much closer, meaning security applications can be much more effective [20]. An important feature of VFC, then, is that decision-making is performed in the local environment, meaning that critical computations can still be performed with low latency even if the link to the Wide Area Network (WAN) is broken.

Furthermore, a more significant advantage that VFC enjoys is its ability to create a much more localized and scalable security mechanism. Rather than relying on a number of different untrusted members of a vehicular cloud, VFC instead relies on a single trusted piece of infrastructure. In fact, even if the RSUs and infrastructure pieces cannot be trusted, recent publications such as [21] have taken advantage of this benefit to propose efficient authentication schemes for VFC that would not be available in VCC, creating a zero-trust framework enabling authenticated fog computing.

3. Literature Review: V2X Data Engineering & Simulations

Given the high volume of data, as well as the different varieties of data, expected in V2X communications, any data engineering platform handling V2X data must be designed to accommodate for high demands in latency and bandwidth. These platforms must have the three components that make up a data management system – data ingestion, data storage, and data analytics [22]. A summary of the three components is shown below, pulled directly from [22].

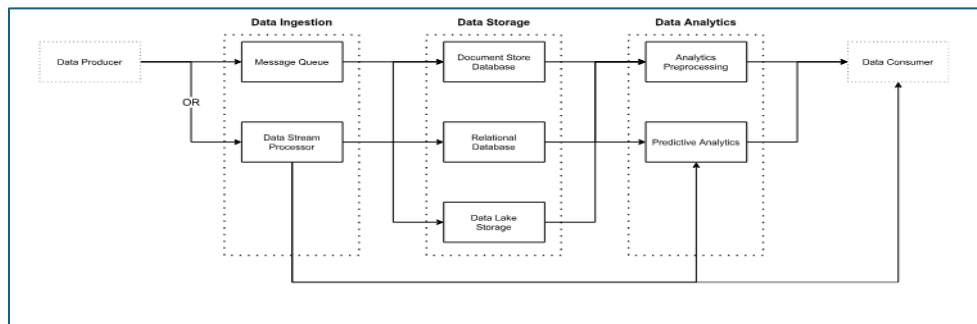


Figure 6: A summary of the components making up a data management system in V2X settings. This diagram is retrieved directly out of [22], which discusses the current state of data engineering in V2X.

3.1: Data Ingestion – MQTT & Kafka

3.1.1: Message Broker - MQTT

In data engineering, a message broker refers to any software that aids the communication and exchange of information between different applications, protocols, and services. In order to achieve this, message brokers use a message queue to store the backlog of incoming messages as receiving applications process the data. These queues have the benefit of maintaining the order of the incoming messages, and some provide guarantees that the messages are not lost or duplicated during their path to the destination applications [23].

A variety of messaging protocols have been used in V2X settings, including Advanced Message Queuing Protocol (AMQP), Constrained Access Protocol (CoAP), and Message Queuing Telemetry Transport (MQTT). An analysis of a variety of these protocols in settings with different numbers and sizes of messages has demonstrated that MQTT performs the best in terms of response times when the number of messages being transferred is high [24]. Because MQTT was designed for machine-to-machine data transfers in low-bandwidth and unstable network connections, not to mention its guarantee on data delivery [25], MQTT-based V2X data ingestion appears to be the best messaging protocol to use in a data platform.

MQTT uses a publish-subscribe communication model, where the publisher and the subscriber never directly interact with each other. Instead, an intermediate link known as the MQTT Message Broker transfers the messages between the two. MQTT clients can subscribe to specific topics at these message brokers, and messages sent to specific topics will be delivered to the relevant subscribers. In V2X settings, connected vehicles can publish their different V2X messages to various topics corresponding to different dataflows, and infrastructure points can also publish processed computations back to the connected vehicles as well. An example of this is shown in the below figure.

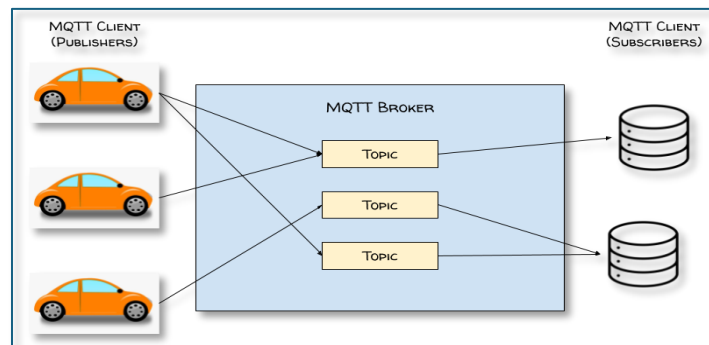


Figure 7: Example of MQTT usage in V2X settings. Here, connected vehicles publish V2X data to topics via the MQTT broker, and data is forwarded to the various MQTT subscribers that use this data in their workflows.

MQTT appears to have been used in a variety of V2X settings. For example, [26] has used MQTT as its messaging protocol, with telemetry servers receiving and forwarding positional and signal strength data from CVs to its big data platform. [25] has also demonstrated using MQTT transmit CV sensor data, citing MQTT's horizontal scalability, network transparency, and ease of integration, all on low-resource systems. Finally, [27] has used MQTT for sending/receiving mobility data for emergency vehicles due to its simplicity in making real-time data available to multiple services.

3.1.2: Message Broker - Kafka

Apache Kafka is an event streaming platform that provides high throughput and low latency, like MQTT. In Kafka, a message broker, which is a Kafka server responsible for storing and handling data, receives the data from producers and forwards it to its destination topics. Data is also partitioned within each topic so that operations can be performed in parallel. Because there is more than one broker running at a given time, a Kafka cluster refers to a group of brokers working together to share and replicate data. A summary of Kafka's architecture is shown in the diagram below.

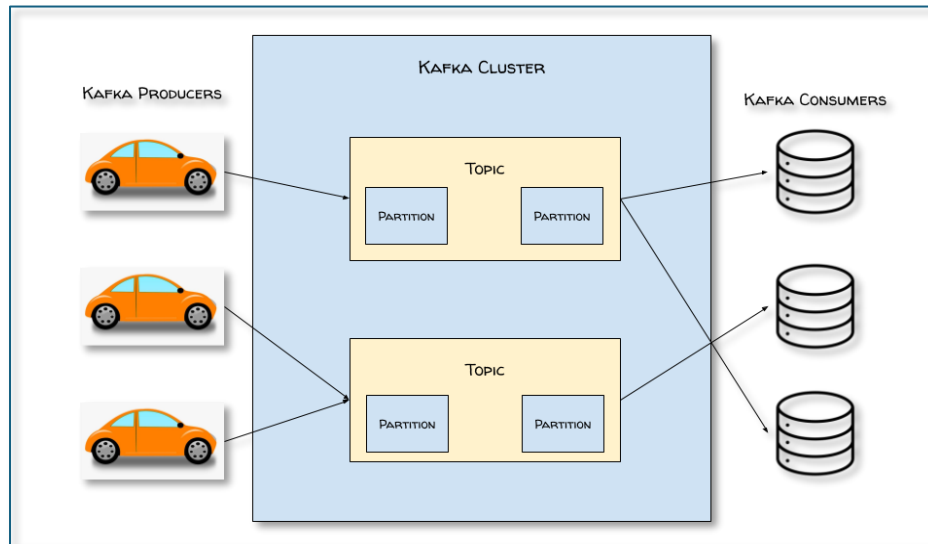


Figure 8: Apache Kafka architecture consists of producers and consumers, with the Kafka cluster consisting of many brokers receiving and forwarding data received from producers through topics. Data is separated into partitions for parallelism.

Works such as [24] have shown that in previous versions of Apache Kafka (pre-4.0), Kafka alone is insufficient for V2X applications. Although Kafka was found to perform the best when it comes to throughput, performance quickly declines once it reaches a certain threshold for the number of partitions. However, Kafka's latency performance was also found to fluctuate, only providing acceptable levels of latency under static conditions. This correlates with the findings in [28], which tested Apache Kafka in various settings and

found that although the highest total average latency was within the US Department of Transportation's recommended system requirements, this latency only considered the message delivery times and not the processing that would be performed. Thus, usage of Apache Kafka appears to be best for environments with rigid settings only, which is not expected in fluctuating road traffic conditions. [29] has found similar results, stating that Kafka alone is insufficient to satisfy the fault-tolerance and horizontal scalability requirements demanded by V2X traffic.

3.1.3: Using MQTT and Kafka Together

A combination of MQTT and Kafka, however, has been demonstrated to perform with better throughput and latency [24]. In fact, researchers have explored using a combination of MQTT and Kafka to better handle the problems of using Kafka or MQTT alone. [29] uses MQTT brokers at the edge, where MQTT source connectors receive messages from CVs, RSUs, and other data providers, then publish them to a Kafka cluster to distribute the data between different processing nodes and databases. Figure 9 shows a combination of MQTT and Kafka in use in this proposed architecture. Simulations on this architecture have found satisfactory performance on Cooperative Awareness Messages (CAMs) when it comes to latency as well as one-to-many and many-to-one guarantees.

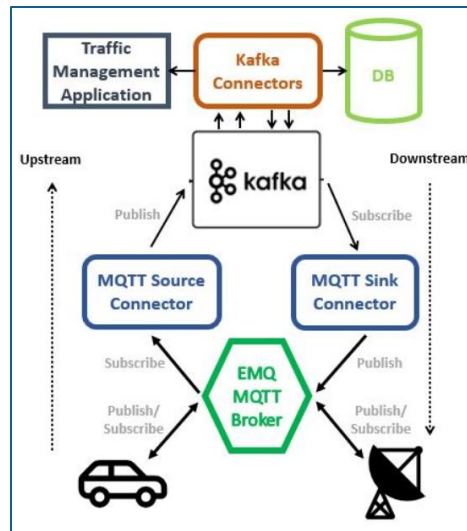


Figure 9: Proposed architecture in [29] integrating in MQTT at the edge and Apache Kafka towards the data processing infrastructure.

The V2X industry has been moving towards adopting a combination of MQTT and Kafka as well. HiveMQ, a company providing enterprise-grade MQTT broker services, promotes combining MQTT and Kafka for real-time data streaming and processing – in fact, an electric vehicle company, Rimac, has used HiveMQ's MQTT broker service and Kafka to improve performance [30]. Confluent has also recommended using MQTT and Kafka for

Internet-of-Things projects, highlighting a V2X example in which maintenance was calculated for 100,000 connected cars [31].

Researchers studying topics outside of V2X settings have found success in combining MQTT and Kafka as well. In [32], ThingsBoard Message Queue (TBMQ), an implementation of the MQTT protocol, was proposed and tested with Kafka to support over 100 million simultaneous connections and a throughput of over 3 million messages per second. On the other hand, researchers in [33] developed an edge computing platform consisting of MQTT and Kafka for a smart farm, finding that integrating both technologies enabled data order to be guaranteed and that experimental response times were under 100 milliseconds. It becomes obvious that many industry experts have identified significant advantages in using a combination of both MQTT and Kafka, suggesting that an optimal data platform for V2X settings should utilize both as a result.

3.2: Data Storage – MongoDB

For the scope of the capstone, the database segment of the data platform will be purposed for temporary storage. A review of existing literature has suggested that MongoDB may be the most optimal database type to be used for a data platform in V2X settings at the fog level.

MongoDB is a database that stores data while maintaining flexibility in the data schema. Just as a traditional database consists of a table composed of rows, MongoDB has documents, which are in a JSON-like structure known as binary JSON (BSON), that make up a collection. This is the most important and relevant feature, as the BSON-formatted documents supports additional data types compared to JSON (dates, object IDs used as unique identifiers, and binary data), enabling a diverse range of unstructured data to be held. Additionally, although MongoDB has sharding capabilities, where database loads are distributed across different machines, the data platform proposed for the capstone will not be able to make full use as it will be hosted on a low-power, low-compute processor on an RSU. Regardless, MongoDB in summary enjoys high performance, scalability, and flexibility, making it an ideal database choice for a V2X application [34]. Each “sensor measurement” – or, in our case, V2X packet – can be stored as separate documents, as suggested by [35], which means other packet data will not be affected and the independence of each document will be preserved.

The literature on the optimal database at the fog level for V2X communications is lacking at the time of writing, with a 2024 review finding that there is an absence of a standard benchmark in comparing databases at the edge. Researchers in this study found that most works in this area were tested on unrealistic workloads, even discovering that “[n]o publication surveyed compared itself against competing solutions” [36]. That being said,

the view determined that relational database management systems are strongly discouraged in V2X settings, as maintaining the location of a car in such databases would incur significant overhead. When combined with [22], which recommends the usage of document stores such as MongoDB for short-term access as well as quick access and fetching capabilities, MongoDB appears to be well-suited for a data platform meant for temporary storage requiring quick access and flexibility to accommodate a variety of different V2X message types.

Fortunately, there have been works that have applied MongoDB in V2X settings – the only item missing, as described above, is the benchmarking to compare the deployment of a certain database management system with other similar works. In [37], authors used MongoDB as one of its database solutions to store metadata, security policies, and complex relationships between different network flows. On the other hand, the Department of Transportation in its Connected Vehicle Pilot Deployment Program in New York City selected MongoDB for its storage engine due to its flexibility in handling structured and unstructured data [38]. It is important to note that this pilot program did not deploy the storage engine at the edge, instead relying on the cloud to take advantage of MongoDB’s sharding features.

Researchers in [39] successfully used MongoDB on edge nodes sitting on RSUs and edge servers to handle processed data to be transmitted back to in-transit mobile edge nodes, although the reasoning for the selection of MongoDB was not included in the work. In fact, the architecture used by [39] is very similar to the one used by this capstone project – a diagram of this team’s data platform can be shown in Figure 10.

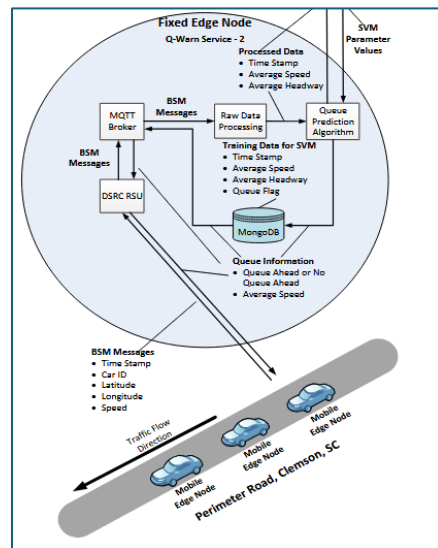


Figure 10: Illustration of [39]’s Q-Warn Service-2 implementation on the outermost layer, which uses MongoDB on an RSU to process and deliver important safety application data.

3.3: Data Analytics – Apache Flink

The data platform for this capstone project contains a data processing segment responsible for performing custom transformations on incoming messages. Because V2X applications are very latency-sensitive, any incoming data should be acted upon immediately, leading to a need for stream processing. Fortunately, some more literature review on this aspect of the database management system exists, with Apache Flink appearing to be the most widely used data processing engine.

Apache Flink is optimized to handle both unbounded and bounded data streams [40], and considering our data platform will be receiving V2X traffic packets continuously, Flink's optimizations for unbounded data streams are a good match. Additionally, Flink is known to be efficient for event-driven applications, which locally stored data is incrementally and asynchronously provided for processing. According to Flink documentation, Flink's support for customization, time control, and savepoints makes it an excellent choice for applications specialized for fraud and anomaly detection [41]. Flink applications consist of a graph of streams and transformations, and Flink's runtime consists of a JobManager, which is responsible for orchestrating the different tasks of the application, and a TaskManager, which executes the smaller tasks making up the graph of transformations.

In fog-based data processing, Apache Flink appears to be the most optimal choice. In [42], not only did researchers conclude that fog-based big data processing was a promising area for latency-sensitive applications, but also that Apache Flink should be recommended as a stream processing framework at the fog level. The authors cited Flink's ability to handle events in a timely manner, even in potentially unreliable environments. Apache Flink's savepoint feature also ensures that fog computing devices, which may fail or suffer from connectivity issues, have a layer of resilience and consistency.

When compared with Apache Spark, [43] found that Apache Flink outputs a seriously higher throughput in controlled experiments – and most importantly, authors cited the applicability and relevance of Flink in lower-compute environments such as a Raspberry Pi 3b+ that only has 1 gigabyte of RAM, 32 GB of storage, and a quad-core 64-bit ARM processor. To that end, it is no surprise that Apache Flink has been adopted by some proposed data platforms, including the DRIVEN framework in [44] that was applied on datasets to achieve a 90% reduction in transmission times.

3.4: VESNOS – A V2X Simulator

VESNOS (Vehicular Secure Network Open Simulator) is an open-source simulation framework designed specifically for cyber-security research in connected and automated vehicle (V2X) domains [3]. Its architecture merges the strengths of multiple specialized

simulation engines to capture realistic vehicle mobility, sophisticated wireless communications, and trusted data exchange.

VESNOS consists of two core components:

- **SUMO (Simulation of Urban MObility).** SUMO provides high-fidelity traffic dynamics, vehicle behaviors, and realistic networked environments. VESNOS leverages SUMO via the TraCI API, enabling real-time control of vehicle movement and interaction with road infrastructure.
- **OMNET++/Veins.** This acts as the backbone for V2V (vehicle-to-vehicle) and V2I (vehicle-to-infrastructure) networking. The Veins framework integrates with OMNET++ to simulate wireless channel properties, complex communication topologies, and dynamic vehicle networks.

The simulator's architecture tightly couples traffic and network layers, so that any change in vehicle position or environment directly influences the corresponding communication events. As a result, the simulator mirrors real-world interdependencies between mobility and networking in intelligent transportation systems.

VESNOS enhances the existing underlying SUMO/OMNET++ components by adding the following:

- **Security Certificates & PKI.** The defining innovation of VESNOS is its integrated Public Key Infrastructure (PKI). Vehicles and roadside units exchange digital certificates and encrypted messages, following best practices modeled after US-DOT standards. This facilitates trustworthy communication, detection and mitigation of cyberattacks (e.g., replay, Sybil, and falsification attacks), and experimentation with security protocols in V2X environments.
- **Customizability & Extensibility.** VESNOS offers extensible modules for both mobility and network simulation, making it suitable for evaluating a variety of scenarios, such as new driving behaviors, different vehicle types, network protocols, and security models.
- **Realistic, Secure Research Environment.** By simulating certificate-based authentication and encrypted communications, VESNOS provides a laboratory-grade setting for research into adversarial threats and resilience in modern V2X systems.

4. Literature Review: Autonomic Cybersecurity and V2X

4.1: Autonomic Cybersecurity

Autonomic computing refers to systems that can manage themselves. This concept requires what are known as self-CHOP properties – self-configuring, self-healing, self-optimizing, and self-protecting [45]. In summary, these types of systems must be able to (1) reconfigure themselves through dynamic allocation of their resources; (2) detect and remediate unanticipated events and failures; (3) continuously monitor themselves to find opportunities for improvement; and (4) detect and defend against potential malicious actions performed against them. Autonomic cybersecurity expands on autonomic computing, applying self-CHOP properties to autonomously detect and recover from cyberattacks, as defined by the authors who proposed the idea of autonomic computing [46].

Today, autonomic cybersecurity is divided into five separate categories: systems that (1) use MAPE-K approaches, (2) do not adopt the MAPE-K approach; (3) directly implement autonomic properties; (4) provide frameworks to support autonomic development; and (5) serve as autonomic cybersecurity development tools [47]. MAPE-K refers to “Monitor, Analyze, Plan, Execute, and Knowledge,” derived from the authors of [46]’s vision in autonomous security of self-managing components interacting with each other to enhance the security of an overall system. Figure 11 demonstrates the interplay among the five principles of this approach.

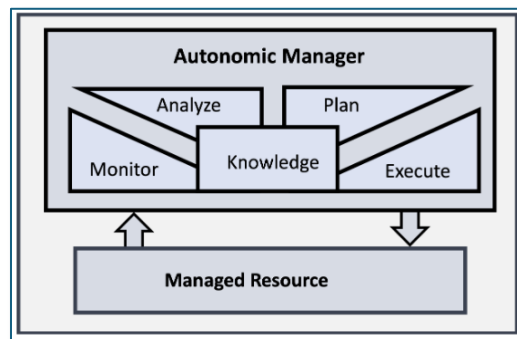


Figure 11: Interplay among the different components of MAPE-K approaches to autonomous cybersecurity as defined in [46]. The figure has been derived directly from [47].

An example of applied autonomic cybersecurity can be found in [48], where small IDSs that independently monitor different components of a smart power grid relay their outputs to a central aggregator responsible for determining if there is a malicious threat and responding with appropriate remediations. It becomes straightforward that all five components of the MAPE-K architecture are present, with the IDSs monitoring and analyzing different components of the smart grid along with the aggregator analyzing,

planning, and executing in response to its findings. This two-tiered nature emphasizes the explainability of the system, enhancing the overall knowledge base as well.

4.2: Autonomic Cybersecurity in V2X

Le Lann first proposed applying autonomic cybersecurity to secure V2X communications in [49], envisioning a self-configuring and self-healing vehicular network that ensures safety, efficiency, privacy, and cybersecurity. At the time of writing, Le Lann noted that existing V2X standards, such as ETSI's technical specifications (TS) on Intelligent Transport Systems (ITS), were not sufficient in providing all four guarantees – notably, privacy and safety. Instead, the author envisioned a future where autonomous vehicles form “cohorts” that demonstrate self-healing and self-protecting properties through mutual authentication.

However, a literature review has revealed that there are no any explicit mappings of the MAPE-K autonomic cybersecurity architecture to V2X settings other than [49], with most works implementing autonomic-like behaviors such as self-organizing and hierarchical/fog coordination. In fact, there are many works that address different components of the MAPE-K architecture in V2X security, especially the Monitor and Analyze aspects, but there appears to be little research on implementing the Plan, Execute, and Knowledge layers. For example, [50] implicitly addresses the Monitor and Analyze components by developing a machine learning-based IDS at the vehicle and RSU level, but this work falls short of actually applying the self-CHOP principles of autonomic cybersecurity to self-configure, optimize, or heal in response to the IDS findings.

However, there are a few works that demonstrate the MAPE-K architecture and self-CHOP principles. [51] demonstrates the self-healing and self-configuring components of autonomic cybersecurity, proposing a new routing protocol that offers automated scoring through fog-based IDS decisions to reconfigure routing and avoid untrusted vehicles. As a result, the work applies all four self-configuring, healing, optimizing, and protecting principles, and the knowledge component of MAPE-K is implicit ingrained into the ensemble of decision tree, random forest, extra trees, and XGBoost models. The only shortcoming is that any new learned knowledge during the application of the model is not fed into the system.

4.3: Security Threats to V2X Communications

V2X security is critical, but also vulnerable to a range of cyberattacks targeting authentication, integrity, confidentiality, and system availability [52]. This section contains brief descriptions of several key threats.

4.3.1: Sybil Attacks

These attacks involve a Sybil attacker who forges multiple fake identities, allowing a single vehicle to masquerade as many. This disrupts traffic management and can manipulate network consensus (e.g., convincing others of false congestion or road incidents), undermining trust in V2X data.

4.3.2: Denial of Service (DoS) Attacks

Denial of service attacks flood the network or roadside units with excessive messages or requests, overwhelming system resources and causing legitimate V2X messages to be dropped. The result is loss of availability, where vehicles may not receive urgent safety alerts, risking public safety.

4.3.3: Evil Twin Attacks

In an Evil Twin attack, a malicious roadside unit or access point impersonates a legitimate one. Vehicles that connect to the fake unit may have their data intercepted, manipulated, or redirected, exposing sensitive information and enabling man-in-the-middle manipulation.

4.3.4: Additional Attack Types

Additional threats include replay attacks (resending valid data to mislead vehicles), GPS spoofing (feeding false location data), jamming (blocking wireless signals), and eavesdropping (silently recording communications for later exploitation).

4.4: Denial of Service Attacks through Basic Safety Message (BSM) Flooding

Denial of service attacks can be perpetrated in any layer throughout the stack. Our capstone, as a result, aims to focus on denial-of-service attacks on the network/transport layers. Our goal is to detect and filter out malicious messages during flooding attacks by employing misbehavior detection. This section briefly reviews some existing literature investigating BSM flooding attacks to achieve denial of service.

In [53], a transport-layer UDP flood attack for 2 minutes was found to have increased latency of BSMs to unacceptable levels, resulting in failures for forward collision warning (FCW) systems on vehicles from issuing alerts. The same research found that application-layer flooding, where valid but oversized BSMs are transmitted, increased latency to double the acceptable level, with FCW alerts being delayed by an entire two seconds. A combination of these attacks was found to increase latency to 5-8 times the acceptable level, with the FCW system completely failing.

In [54], a SYN flood attack (TCP) was demonstrated, with the intent of disrupting a Stop Sign Gap Assist (SSGA) application that broadcasts information from an RSU that warned drivers of potential collisions at stop sign intersections. The effectiveness of the SSGA

application relies on the availability of nearby cars – for example, the speed and acceleration of nearby vehicles. By performing a SYN flood attack consisting of BSMs to increase the computation of safety operations of the SSGA application, legitimate BSMs were not received, leading to safety failures.

4.5: Detecting Anomalous and/or Malicious BSMs

There has been plenty of research into detecting anomalies within BSMs to identify malicious packets and mitigate adversary actions. For the purposes of this capstone project, only application-level features will be used for anomaly detection, meaning this section’s review is limited to data contained within BSMs only. The data contained in BSMs are defined by the SAE J2735 standard, as described in [Section 2.2.1](#).

In addition to the raw data fields contained in BSM messages, [55] groups some of the features together in order to use the relationships between fields such as the yaw rate (steering wheel angle) and acceleration rate to ensure values are realistic. BSMs can also be grouped together by their *TemporaryID* field, which changes every 5 minutes per vehicle, to check for consistency in these relationships.

Among the different features used by [50] to detect anomalies, researchers focused on the timing of incoming BSMs, looking for the latency the frequency of communications as indicators for suspicious behaviors. Similarly, [56] analyzed the time between two subsequent message occurrences from the same source, as well as inter-arrival times within windows of predefined interval sizes. In addition, the BSM as a whole was inspected to see if it complied with the J2735 standard.

5. Technical Solution, Design & Analysis

5.1: V2X Data Platform

This capstone project aims to design a lightweight, modular platform that addresses the shortcomings of vehicular fog computing (VFC) environments without compromising the essential capabilities necessary for real-time data processing and security analysis. The platform consists of three essential layers: the message ingestion layer, the storage layer, and the processing layer, all of which are managed through a Python-based control framework.

Although this portion of the capstone project serves to demonstrate that the technologies that we selected and the final data platform we developed can be applied as an efficient and lightweight data platform deployable at the fog level, it is important to consider that in order for the platform to be applied in full autonomous V2X computing as proposed in [49], it must be capable of both publishing and subscribing messages. In other words, it should support a bidirectional flow of data, both from the CVs/RSUs to the platform and from the

platform to the CVs/RSUs. This will further enable the platform to be used for autonomic cybersecurity as introduced in [Section 4.2](#). However, for the purposes of the capstone, only one direction has been implemented and tested, where the platform receives data from the edge nodes. Implementation of the other direction is future work that should be straightforward.

The final version available for the capstone deliverables consists of three components – a combination of MQTT and Apache Kafka as the message broker, MongoDB for the database, and Apache Flink for the data processing engine. The data platform is summarized in Figure 12.

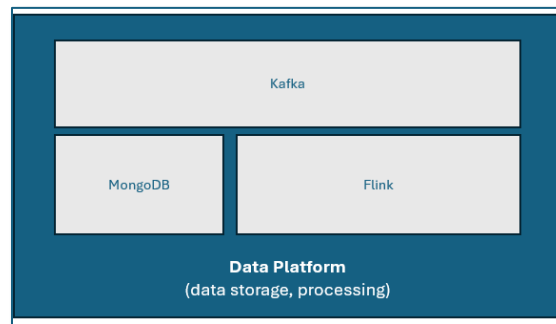


Figure 12: High-level overview of the efficient and lightweight data platform proposed by this capstone project to be deployed at the fog layer.

The specifications of the data platform are:

- Python v3.13.2
- Data broker: *confluent-kafka*==2.12.2, *paho-mqtt*==2.1.0
 - Mosquitto v2.0.12
 - Confluent v4.45.1 for Apache Kafka
- MongoDB 7.0
- Apache Flink v1.18.0.

5.2: Component Selection and Rationale

5.2.1: Message Broker

Mosquitto as the MQTT broker and Apache Kafka as the secondary message broker are used by the platform. The double brokering strategy with MQTT as a supplement to Kafka was selected for improved latency and scalability, as recommended by research outlined in [Section 3.1.3](#).

Connected vehicles that would be transmitting messages are represented as MQTT clients that publish V2X messages, such as BSMs. *paho-mqtt* is used as the Python library for MQTT. Version 5 of the MQTT protocol was used over version 3.1, as the latest version

enjoys new features such as shared subscriptions, custom user key-value properties, and session expiry [57].

```
def publish(self, topic, payload, qos, props=None):
    """
    Wrapper function for the underlying paho-mqtt client's publish() function.

    :param topic: String. Topic to which the client publishes.
    :param payload: Bytestring. Data to be published to the topic, from which subscribing clients receive.
    :param qos: Integer. Quality of Service. 0 = no guarantees, 1 = at least once, 2 = exactly once.
    :param props: List of key-value pairs containing properties associated with the data platform.
    """
    # Check if the client is connected first!
    if not self.is_connected:
        print(f'[PUBLISHER {self.client_id}] Tried to publish without connecting. Exiting.')
        return

    # Keep track of latencies using UserProperties, as well as other per-message metrics
    if not props:
        props = Properties(PacketTypes.PUBLISH)
        props.UserProperty = []

    props.UserProperty.extend([
        ("topic", topic),
        ("client_id", self.client_id),
        ("publish_ts", str(time.time()))
    ])

    result = self.client.publish(
        topic=topic,
        payload=payload,
        qos=qos,
        properties=props
    )

    self.mid_topics[result.mid] = topic

    return result
```

Figure 13: Code snippet showing the initialization of the MQTT client publishing messages to the bridge, with custom user properties to measure latency.

V2X messages are processed by a bridge that acts as both an MQTT subscriber and a Kafka producer. *confluent-kafka* was selected as the Python library for Apache Kafka instead of *kafka-python*, as *confluent-kafka* is much more performant due to its implementation in C rather than being implemented purely in Python from the ground up [58]. The bridge forwards the message received from the MQTT client, producing it for the designated Kafka topic.

The other applications of the data platform – MongoDB and Flink – act as Kafka consumers, receiving the messages forwarded by the Kafka broker.

```

def _on_message(self, mqtt_client, userdata, msg):
    """
    Overrides the on_message function contained in MQTTClient to forward the
    received messages from MQTT to Kafka.
    """
    self.received_count += 1
    if DEBUG_MODE and self.received_count % 500 == 0:
        print(f'Bridge received message. Total # of messages: {self.received_count}')

    # Find the MQTT topic ID, and use the mapping dictionary to find
    # which Kafka topic the message should be forwarded to
    kafka_topics = self.resolve_kafka_topics(msg.topic)

    send_time = time.time()

    # Use Kafka headers to keep track of latency between Kafka producer and consumer
    # Total latency from MQTT publisher to Kafka consumer can be calculated using
    # the payload itself - refer to MQTTClient.py to see how the payload's first
    # X bytes represent the transmit time by the MQTT publisher
    try:
        for kafka_topic in kafka_topics:
            self.kafka_producer.produce(
                topic=kafka_topic,
                value=msg.payload, # TODO: decode('utf-8')?
                key=kafka_topic.encode(),
                on_delivery=MQTTKafkaBridge.delivery_callback,
                headers=[
                    ('kafka_send_ts', str(send_time).encode('utf-8'))
                ]
            )
            self.produced_count += 1

        self.kafka_producer.poll(0)
    except BufferError as e:
        print(f'Error: Bridge encountered BufferError. {e}')

    if DEBUG_MODE and self.produced_count % 500 == 0:
        print(f'Bridge produced message to Kafka. Total # of messages: {self.produced_count}')

    # Retain the latency calculations and print statements in the parent class
    super()._on_message(mqtt_client, userdata, msg)

```

Figure 14: The `_on_message` function of the bridge that forwards received MQTT messages to Kafka consumers.

5.2.2: Storage - MongoDB

For storage, we used MongoDB (v7.0) as our database system. The literature review contained in [Section 3.2](#) explores why researchers in different works adopted MongoDB over other database types, but in this section, we explain our reasoning behind our selection.

First, the semi-structured nature of V2X messages, which contain fields that are variable depending on message type and installed sensors, fits MongoDB's document-oriented model. Second, MongoDB's aggregation framework comes with built-in data transformation and analysis capabilities to complement our processing layer [59]. Third, MongoDB Community Edition's lightweight deployment profile enables deployment on RSU hardware with restricted resources but still accommodates essential features like indexing for high-performance time-series queries and authentication for secure multi-tenancy [60] [61].

```

def create_indexes(self):
    """Create indexes for efficient V2X data queries"""
    try:
        # BSM (Basic Safety Message) collection indexes
        bsm_collection = self.db['bsm_messages']
        bsm_collection.create_index([('timestamp', DESCENDING)])
        bsm_collection.create_index([('vehicle_id', ASCENDING),
                                     ('timestamp', DESCENDING)])
        bsm_collection.create_index([('simulation_time', ASCENDING)])

        # Attack events collection indexes
        attack_collection = self.db['attack_events']
        attack_collection.create_index([('timestamp', DESCENDING)])
        attack_collection.create_index([('attack_type', ASCENDING),
                                         ('timestamp', DESCENDING)])

        logger.info("MongoDB indexes created successfully")
    except Exception as e:
        logger.error(f"Failed to create indexes: {e}")
        raise

```

Figure 15: Snippet from the data platform showing MongoDB index creation for V2X queries

```

def insert_messages_batch(self, collection_name: str, messages: List[Dict]):
    """Insert multiple messages in batch for efficiency"""
    if not messages:
        return

    try:
        collection = self.db[collection_name]
        # Add insertion timestamp to all messages
        for msg in messages:
            msg['inserted_at'] = datetime.utcnow()

        result = collection.insert_many(messages, ordered=False)
        logger.info(f"Inserted {len(result.inserted_ids)} messages into {collection_name}")
        return result.inserted_ids
    except BulkWriteError as bwe:
        logger.warning(f"Bulk write completed with errors: {bwe.details}")
    except Exception as e:
        logger.error(f"Failed to insert message batch: {e}")
        raise

```

Figure 16: Code snippet showing batch insert for high throughput.

5.2.3: Processing - Apache Flink

The data processing layer utilizes Apache Flink (v1.18.0) for real-time analytics and stream processing. Flink was preferred because its event-time processing semantics handle out-of-order message arrival correctly, a common occurrence in wireless V2X networks where messages are subject to variable latency. Flink's stateful stream processing allows the implementation of sliding window operations necessary for temporal attack detection patterns, like detecting denial-of-service attacks via abnormal message rate analysis. We use PyFlink, the Python API of Apache Flink, to ensure consistency with our platform's Python-focused architecture [40] [62] [63].

5.2.4: Python Orchestration Framework

The platform components are integrated through a Python orchestration layer on top of Python 3.11. This framework manages the lifecycle for all platform services and configurations, along with a REST API for external integration. Key modules are:

- **Configuration Manager:** Unified management of platform parameters like Kafka topic definitions, MongoDB connection strings, and Flink job specifications
- **Service Registry:** Automatically restarts and health-checks platform components

- **Data Pipeline Manager:** Automatically manages the creation of defined Flink processing pipelines from researchers for their analysis tasks
- **Model Deployment Interface:** A standardized API to deploy trained ML models as Flink operators for performing inference in real time

5.3: Detecting Denial of Service & Fabrication Attacks

We now apply the data platform developed in the previous sections to autonomic cybersecurity in the context of V2X. As mentioned in [Section 5.1](#), the current implementation of the data platform is unidirectional, making it incomplete for the “autonomic” aspect, but achieving bidirectional capabilities for self-healing and self-configuring operations will be straightforward.

5.3.1: Threat Model – BSM Flooding

Our threat model that we tackle in the scope of the capstone project, as described briefly in [Section 4.4](#), is BSM-flooding denial of service attacks. As seen in [53] and [54], adversaries can flood target V2X safety applications with BSMs to completely deny safety communications crucial for autonomous driving. Thus, for the purposes of this capstone project, we address scenarios where adversaries flood the VANET with BSMs to deny the availability of important safety services, such as FCW alerts in CVs and SSGA broadcasts from RSUs.

We address three types of flooding attacks – transport layer TCP flooding attacks, transport layer UDP flooding attacks, and application layer flooding attacks consisting of differently-sized, protocol-compliant BSMs. The goal would be to filter out malicious BSMs during the floods so that (1) only the legitimate BSMs are processed by safety applications on RSUs, or (2) legitimate BSMs are notified to the destination CVs during active denial-of-service attacks.

Although the first example focuses on V2V communications with BSMs, detection and notification can occur on the infrastructure side as well. We know that BSMs are not intended for RSUs, but they can overhear V2V traffic, with V2V beacons especially transmitted in plaintext (i.e. unencrypted). In fact, RSUs act as local collectors or relays for security credential management or misbehavior detection [64]. As a result, our capstone project is still very relevant – having an intrusion detection system that detects misbehavior, filters out the non-legitimate BSM in an active transport or application layer flooding attack, then transmits the legitimate BSM back to the target vehicles would be a solution to this problem from the first example. In the second example, the SSGA application would not be overloaded by only allowing BSMs that our platform assesses to be legitimate.

5.3.2: Autonomic Cybersecurity Approach for BSMs

For a future with fully autonomous driving, the security stack must behave as an autonomic cybersecurity system operating at the fog level, close to the vehicles and RSUs. The goal of the proposed IDS is to detect denial-of-service (DoS) and fabrication attacks against the VANET at the fog layer, generate timely alerts for other connected vehicles, and trigger local responses if required.

This design aligns with Le Lann’s vision of autonomic vehicular cybersecurity, where trust is shifted to local environments, exposure is minimized to short-range communications, and malicious nodes are detected and excluded locally with timing treated as a security primitive.

The IDS adopts a three-tier hybrid architecture composed of multiple micro-IDS components and a final aggregator.

1) Tier 1 – Time Interval Monitoring IDS (Time IDS).

- a) Focuses on timing intervals between messages from each sender to detect DoS and availability-related anomalies on the BSM channel.
- b) If an improper time interval is detected (e.g., abnormal rate or burst), the packet is forwarded directly to Tier 3 for DoS classification; if the time interval is proper, the packet proceeds to Tier 2.

2) Tier 2 – Motion IDS (Speed, Position, Heading, Acceleration).

- a) Consists of specialized micro-IDSs for speed, position, heading, and acceleration, each responsible for checking physical consistency and motion realism.
- b) Each micro-IDS flags its verdict as “proper” or “improper” and forwards the combined motion verdicts to the aggregator.

3) Tier 3 – Aggregator (Decision Tree).

- a) Implements a decision tree that fuses the outputs of Tier 1 and Tier 2 to produce a final, explainable verdict.
- b) If the Time IDS flags a DoS condition, the aggregator classifies the behavior as DoS directly; if time is normal but motion IDSs flag problems, the aggregator classifies the traffic as fabrication.

This structure combines rule-based checks at the micro-IDS level with autonomic decision logic in the aggregator, providing both robustness and interpretability.

Time is treated as a first-class signal because availability attacks on the entire channel (DoS) are generally more damaging than fabrication by a single vehicle. As a result, the architecture is explicitly optimized to detect timing anomalies very reliably in Tier 1 before examining motion integrity in Tier 2.

Using multiple specialized detectors for time, speed, position, heading, and acceleration yields broader coverage and higher accuracy than a single monolithic IDS. Each micro-IDS behaves like an interpretable engine, exposing which check fired and why, which makes the system easier for operators to trust, debug, and tune over time.

The hierarchical design is also scalable: new attack types can be supported by training small additional micro-IDSs and updating simple aggregation rules, rather than retraining and redeploying a large, opaque IDS model.

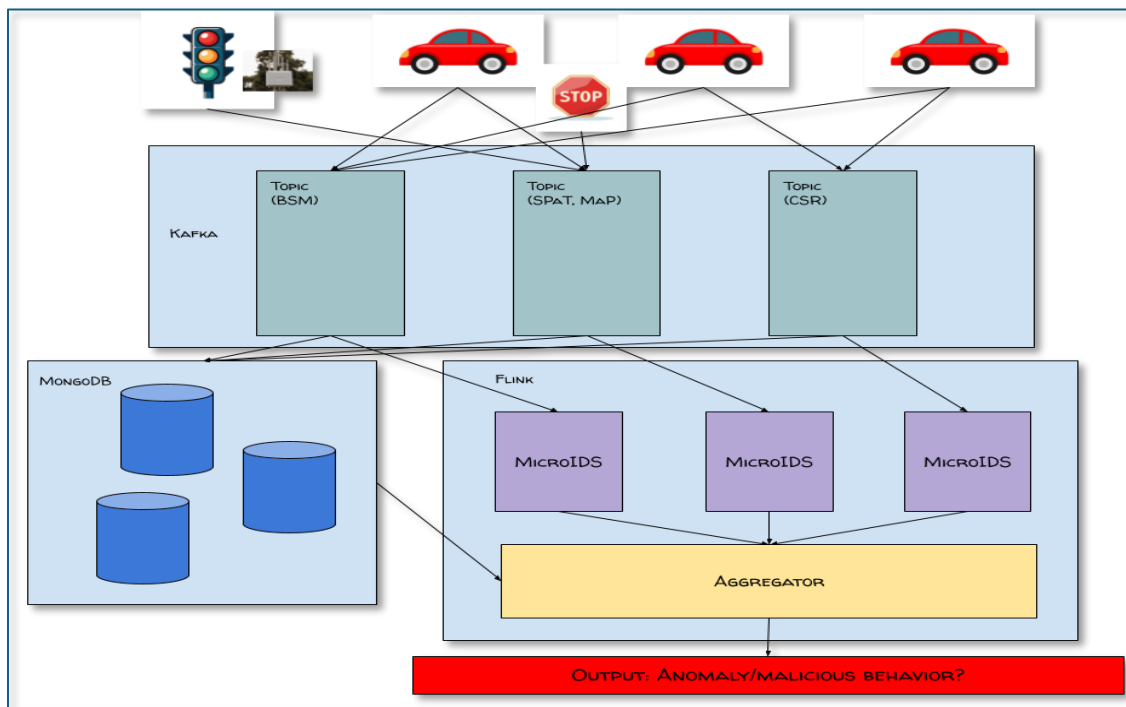


Figure 17: High-level architecture diagram showing how the proposed data platform.

5.4: Integration with the V2X Data Platform

The IDS architecture is designed to integrate directly with the fog-level data platform built on Kafka, MongoDB, and Apache Flink.

- Different V2X message types (e.g., BSM, SPaT, CAM) are mapped to separate Kafka topics, enabling per-type micro-IDS processing pipelines.
- Each micro-IDS subscribes to relevant topics and runs as a lightweight Flink operator, performing real-time anomaly detection on streaming messages.

- Micro-IDS outputs and per-sender state persisted in local MongoDB collections for auditing, explainability, and potential feedback into the Knowledge component of the autonomic loop.

The aggregator consumes the micro-IDS verdicts via dedicated Kafka topics and applies the decision tree to classify behavior as benign, DoS, or fabrication, enabling downstream actions such as dropping malicious packets or notifying higher-level applications.

6. Experimentation, Evaluation & Result Analysis

6.1: Data Platform – Stress Testing

In the earlier phase of the capstone, we evaluated multiple MQTT protocol implementations under V2X-representative workloads, measuring end-to-end latency for messages published at various rates. These workloads were derived from expected metrics calculated in [12], as summarized in Figure 3. In order to stress test the data platform, we simulated different conditions in different environments, ranging from average expected conditions to the maximum expected conditions in urban, suburban, and rural areas:

- Baseline test: 25 clients, 1000 msgs/client, 4096 bytes
- Urban (max) test: 100 clients, 70 msgs/client, 4330 bytes
- Urban (avg) test: 80 clients, 70 msgs/client, 4330 bytes
- Suburban (max) test: 40 clients, 94 msgs/client, 4330 bytes
- Suburban (avg) test: 25 clients, 94 msgs/client, 4330 bytes
- Rural (max) test: 10 clients, 140 msgs/client, 4330 bytes
- Rural (avg) test: 5 clients, 140 msgs/client, 4330 bytes

For MQTT broker performance, our tests showed that Mosquitto (v2.0.12) reported the best latencies and throughput metrics compared to RabbitMQ (v4.2.2, with Erlang OTP v28), NanoMQ (v0.24.6), and HiveMQ (v2025.5). Only the community edition of HiveMQ was tested, as the production-grade version requires a purchase – and the free trial of this enterprise version includes extra convenience features such as a web interface that adds overhead to the overall process. The minimum, median, average, and maximum latencies in milliseconds are attached as separate JSON files as deliverables of this capstone, but a plot of average latencies is embedded in Figure 18. As a result, we have decided to use Mosquitto as the MQTT broker as opposed to the other solutions.

Latency was measured as the time the message was published by the connected vehicle (i.e. the MQTT clients) and the time the message was received by the Kafka consumer. The

publish timestamp was encoded into the message payload to be compared against by the consumer.

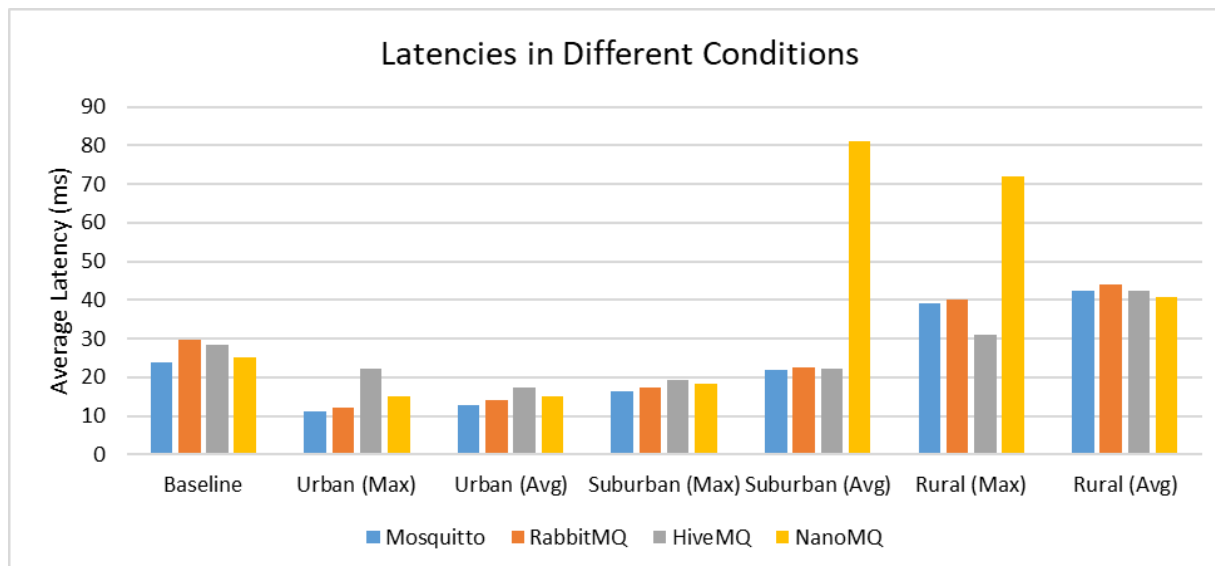


Figure 18: Comparison of overall latencies across Mosquitto, RabbitMQ, HiveMQ, and NanoMQ.

Despite its lightweight and performant nature, NanoMQ appears to have surprisingly struggled with suburban/rural environments. However, a closer analysis reveals that this might be due to an incompatibility or inefficient crosstalk between the MQTT broker and the Kafka broker. When looking at just the MQTT latencies, which is calculated by the difference in time when the message was published and the time the MQTT/Kafka bridge receives the message, the numbers are relatively similar across the four tested MQTT brokers. In fact, looking at solely the MQTT latencies suggests that Mosquitto is the most efficient and most effective MQTT broker to be used across the four tested brokers.

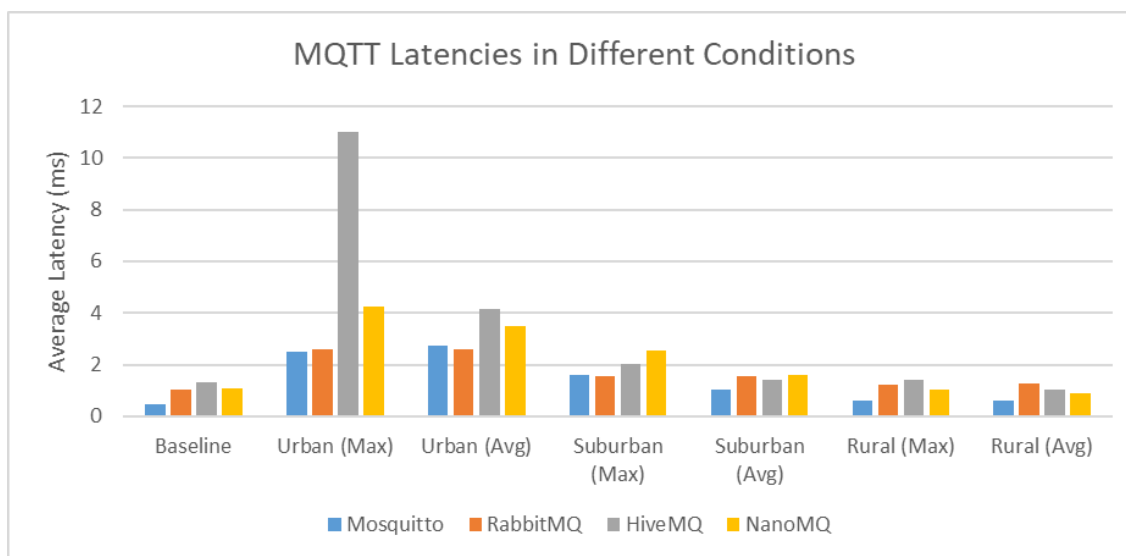


Figure 19: Comparison of MQTT latencies across Mosquitto, RabbitMQ, HiveMQ, and NanoMQ.

Additionally, the MQTT latency data makes it obvious that the Kafka component of the message ingestion piece is the most expensive operation. Although the configurations of both the Kafka producer and consumer can be tinkered with to achieve even lower latencies, our team found that the configurations that we deployed were the most effective during our testing.

To achieve these latencies, on the producer side, the *linger.ms* value was set to 0 so that the maximum time for the Kafka-MQTT bridge to wait before transmitting the received message would be zero (i.e. instantaneous forwarding). The *compression.type* was set to *none* in order to reduce any overhead and compute time caused from compression and decompression of data. Finally, *acks* were set to *all* in order to ensure that there is a guarantee of at-least-once delivery from the bridge to the Kafka consumer.

On the consumer side, the *auto.offset.reset* value was set to *latest* to ensure that older, out-of-context messages were not being processed when newer messages were arriving. Additionally, *fetch.min.bytes* was set 1, and *fetch.wait.max.ms* was set to 5 in order to prevent any potential blocking loops from preventing instantaneous data processing. To this end, both the producer and the consumer has a poll time of 0 as well to avoiding blocking and introducing any latencies at the application-level. We have not experimented with adding/removing backoff times in case of an absence of a message, which may be necessary for low-resource RSUs. Finally, the *enable.auto.commit* value was set to *True*, allowing the consumer to automatically commit its offset (i.e. its progress) to the Kafka broker.

On the broker side, *num.replica.fetchers* was set to 2 in order to align with best practices described in an Apache Kafka white paper in reducing latency as much as possible [65]. This refers to the number of fetcher threads for the broker – as long as there are enough resources on the device running the Apache Kafka broker, this value can be increased to enjoy lower latencies and greater throughput. Additionally, in order to prevent older, out-of-context messages from being stored by the broker, we set the *retention.ms* value to 500, running on the assumption that anything greater than .5 seconds is unacceptable due to the Department of Transportation’s required latency threshold of 1000 milliseconds.

```

class KafkaConsumer:
    def __init__(self, hostname, port, group_id):
        self.hostname = hostname
        self.port = port
        self.group_id = group_id

        self.consumed_count = 0
        self.latencies = [] # Kafka latency
        self.total_latencies = [] # MQTT + Kafka latency

    def connect(self, topics, offset_reset='latest'): # earliest
        config = {
            'bootstrap.servers': f'{self.hostname}:{self.port}',
            'group.id': self.group_id,
            'auto.offset.reset': offset_reset,
            'enable.auto.commit': True,
            'fetch.min.bytes': 1,
            'fetch.wait.max.ms': 5, # ORDER OF WORDS IS DIFFERENT
        }

        self.consumer = Consumer(config)

```

Figure 20: Configuration settings for the Kafka consumer.

```

def kafka_connect(self):
    # Create the configuration for the producer
    config = {
        'bootstrap.servers': f'{self.kafka_hostname}:{self.kafka_port}',
        'linger.ms': 0,
        'acks': 'all',
        'compression.type': 'none',
    }

    self.kafka_producer = Producer(config)

```

Figure 21: Configuration settings for the Kafka producer, which acts as a bridge between MQTT and Kafka.

6.2: Baseline Decision Tree Model Testing for V2X IDS: Phase 1 Evaluation

6.2.1: Introduction and RSU Detection Context

This section presents the design, development, and benchmarking of a baseline Decision Tree model engineered explicitly for roadside unit (RSU) deployment in V2X environments. The sole purpose of this system is to detect potentially malicious Basic Safety Messages (BSMs) delivered to the RSU, rapidly flagging packets that may pose safety or integrity risks within a vehicle communication network. This model is conceived as a technical foundation for building likely efficient MicroIDS+Aggregator architectures for V2X network endpoints.

6.2.2: Dataset Structure and BSM Generation Logic

To ensure direct compatibility with industry standards and real-world deployments, the dataset chosen for training emulates BSM messaging fields and event triggers [66]. Though the underlying dataset was originally structured for CAM simulation, all fields and generation logic used here mirror BSM content and rules [67]. For detection modeling, packets follow transmission intervals (100–1000 ms) and generation triggers based on position, heading, and speed changes typical of BSM protocols. Attack traces include both rapid flooding (denial-of-service) and fabrication of movement parameters, replicating realistic adversarial threats faced by RSUs in operational V2X environments.

6.2.3: Streaming Feature Engineering for RSU Efficiency

The model's engineering pipeline is tightly aligned with RSU hardware requirements. Every packet is processed one at a time, with the system maintaining per-sender state strictly in memory. Nine features—interval between packets, changes in position, heading, speed, raw acceleration, acceleration consistency, and three rule violation flags—are computed instantly and require no batch processing or large statistical windows. This approach guarantees minimal memory usage and instantaneous anomaly detection, which are essential for RSUs expected to process thousands of packets per second.

6.2.4: Model Training, Validation, and Metrics

Robustness was confirmed using seven-fold cross-validation, where each city map in the dataset was held out as a distinct validation set while the others trained the model. The Decision Tree classifier was selected for its hardware efficiency, deterministic decision-making, and transparent logic—vital for regulatory and technical auditability on roadside infrastructure. All nine engineered features were provided as model input. During testing, the system achieved a mean accuracy of 98.85%, detecting a high proportion of malicious BSMs with 86.97% recall and balancing false alarms (precision 73.31%). Average inference time measured just 0.0009 ms per packet and model storage requirements were negligible (14.4 KB). The model was hosted on a 12GB RAM CPU for testing but will be later deployed on Raspberry pi system to gauge actual processing capabilities. These results demonstrate both the reliability and practicality of real-time RSU deployment, with sample confusion matrices confirming thousands of correct detections and very few missed attacks or false positives for each test city.

6.2.5: Significance for MicroIDS + Aggregator Development

This baseline model is far more than a research artifact; it is a direct engineering benchmark for every subsequent IDS strategy to be deployed at the RSU. The strengths—minimal latency, high accuracy, real-time compatibility, and efficient memory footprint—are now the quantitative standards future MicroIDS+Aggregator models must meet or exceed. The value of this benchmark is especially clear in the next phase, where multiple specialized MicroIDS will be developed to detect specific attack types or feature anomalies and their outputs will be fused by an aggregator meta-model. Metrics and computational profiles reported here provide precise targets for accuracy, recall, precision, and resource consumption in all future deployments.

6.3 Micro-IDS + Aggregator Model Results

The micro-IDS detectors were trained and evaluated on simulated CAM data constrained to BSM-equivalent fields, with traffic patterns and attack traces that include both DoS

flooding and fabricated motion parameters. The table below summarizes the per-detector metrics and resource characteristics.

Metric	Time IDS	Speed IDS	Heading IDS	Acceleration IDS	System-level (current)	Expected after optimization
Accuracy	99.48%	99.95%	99.98%	99.52%	–	–
Precision	99.50%	99.92%	99.98%	99.95%	–	–
Recall	99.48%	99.96%	99.98%	99.52%	–	–
F1-score	99.48%	99.94%	99.98%	99.76%	–	–
Latency (ms)	1.6142	1.6142	1.6083	1.6625	1.8 – 6.5	~1.4 (with multithreading)
Throughput (pkt/s)	582	589	591	575	160	700
Memory / vehicle (bytes)	1,358 per vehicle	–	–	–	Local storage footprint scales linearly with vehicles	Target under 500 MB per fog node

Figure 22: Table of per-detector metrics, as well as their expected resource usage characteristics.

These results show that each micro IDS achieves very high accuracy (99.48–99.98%) with tight latency bounds (~1.6 ms per detector) and high per detector throughput, while maintaining a small per vehicle memory footprint suitable for fog level deployment. The current integrated system operates between 1.8 and 6.5 ms in end to end IDS latency at about 160 packets per second, with optimization and multithreading expected to reduce latency toward ~1.4 ms and raise throughput to approximately 700 packets per second.

The models are currently trained and evaluated on simulated data; performance metrics are expected to shift when deployed on real V2X traffic at RSUs, but the architecture and resource measurements demonstrate feasibility for real-time fog deployment.

7. Conclusion

7.1: Discussion

The results from the RSU-centric data platform show that Apache Kafka, MongoDB, and Apache Flink can be composed into a lightweight pipeline that satisfies fog-level constraints for V2X traffic while still supporting realistic workloads derived from SUMO/VESNOS simulations. Latency and packet delivery rate remain within safety-relevant bounds across urban, suburban, and rural profiles, indicating that moving

data engineering functionality from centralized cloud resources to RSUs is both feasible and beneficial for availability. These findings directly address concerns in the literature about cloud outages and WAN dependencies for safety-critical V2X services.

The micro-IDS + aggregator architecture further demonstrates that autonomic cybersecurity concepts can be instantiated in a practical, explainable IDS for BSM-like traffic at the fog layer. By decomposing detection into specialized time and motion detectors and fusing their outputs through a decision tree, the system achieves very high accuracy (99.48–99.98%), millisecond-scale per-detector latency, and a small per-vehicle memory footprint suitable for RSU hardware. This design operationalizes several self-CHOP and MAPE-K properties, supporting local, timing-aware decision-making that aligns with Le Lann’s vision of moving trust and detection into the local vehicular environment.

7.2: Future Work

Future work should first focus on fully integrating the three-tier micro-IDS + aggregator stack into the production data platform path so that BSMs and other V2X messages are inspected in situ as they traverse Kafka topics and Flink jobs. This includes extending the currently unidirectional pipeline into a bidirectional architecture where RSUs can not only detect misbehavior but also publish alerts, blacklist recommendations, or sanitized data back to vehicles and infrastructure, enabling self-healing and self-configuring behaviors at the fog layer.

A second priority is to validate the IDS on real Hopkins V2X testbed data, including on-road BSM-like traffic and additional sensor streams, in order to stress the models under real-world noise, hardware jitter, and heterogeneous radio conditions. Such experiments will refine thresholds, verify that latency and memory budgets remain acceptable at scale, and inform whether additional micro-IDS components are needed for other message types such as SPaT, CPM, or DENM. Over time, the Knowledge component of the MAPE-K loop could incorporate operator feedback or online learning, allowing the system to evolve its rules as new attack patterns emerge.

References

- [1] A. Ohnsman, "Waymo's Robotaxis Are Hitting the Highway, A First for Self-Driving Cars," *Forbes*, 8 January 2024. [Online]. Available: <https://www.forbes.com/sites/alanohnsman/2024/01/08/waymos-robotaxis-are-hitting-the-highway-a-first-for-self-driving-cars/>. [Accessed 10 October 2025].
- [2] M. A. Khan, H. El Sayed, S. Malik, T. Zia, J. Khan, N. Alkaabi and H. Ignatious, "Level-5 Autonomous Driving—Are We There Yet? A Review of Research Literature," *ACM Computing Surveys (CSUR)*, vol. 55, no. 2, pp. 1-38, 2022.
- [3] A. Abdo, G. Wu and N. Abu-Ghazaleh, "VEhicular Secure Network Open Simulator (VESNOS): A Cyber-security oriented co-simulation platform for connected and automated driving," in *SIGSIM-PADS '24: Proceedings of the 38th ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*, Atlanta, 2024.
- [4] A.-M. Corvin, "Cloudflare, Azure, AWS: the striking pattern behind the outage cluster shaking the internet," *CyberNews*, 19 November 2025. [Online]. Available: <https://cybernews.com/cloud/cloudflare-azure-aws-striking-pattern-outage/>. [Accessed 21 November 2025].
- [5] M. T. Kawser, S. Fahad, S. Ahmed, S. S. Sajjad and H. A. Rafi, "The Perspective of Vehicle-to-Everything (V2X) Communication," *International Journal of Computer Science and Network Security*, vol. 19, no. 4, pp. 146-155, 2019.
- [6] M. J. Khan, M. A. Khan, A. Beg, S. Malik and H. El-Sayed, "An overview of the 3GPP identified Use Cases for V2X Services," *Procedia Computer Science*, vol. 198, pp. 750-756, 2022.
- [7] L. Bréhon–Grataloup, R. Kacimi and A. L. Beylot, "Mobile edge computing for V2X architectures and applications: A survey," *Computer Networks*, vol. 206, 2022.
- [8] A. Fitah, A. Badri, M. Moughit and A. Sahel, "Performance of DSRC and WIFI for Intelligent Transport Systems in VANET," *Procedia Computer Science*, vol. 127, pp. 360-368, 2018.
- [9] C. Mellor, "Autonomous vehicle data storage: We grill self-driving car experts about sensors, clouds...and robo taxis," *Blocks & Files*, 3 February 2020. [Online]. Available: <https://blocksandfiles.com/2020/02/03/autonomous-vehicle-data-storage-is-a-game-of-guesses/>. [Accessed 10 October 2025].
- [10] G. G. N. Ali, M. N. Sadat, S. Miah, S. A. Sharief and Y. Wang, "A Comprehensive Study and Analysis of the Third Generation Partnership Project's 5G New Radio for Vehicle-to-Everything Communication," *Future Internet*, vol. 16, no. 1, 2024.
- [11] S. International, "J2735_202409 - V2X Communications Message Set Dictionary," 15 September 2024. [Online]. Available: https://www.sae.org/standards/j2735_202409-v2x-communications-message-set-dictionary. [Accessed 1 September 2025].
- [12] C. 2. C. C. Consortium, "Position Paper on Road Safety and Road Efficiency Spectrum Needs in the 5.9 GHz for C-ITS and Cooperative Automated Driving," 28 February 2020. [Online]. Available: https://www.car-2-car.org/fileadmin/documents/General_Documents/C2CCC_TR_2050_Spectrum_Needs.pdf. [Accessed 12 June 2025].
- [13] N. H. T. S. A. (NHTSA), "V2X Cellular Solutions," October 2016. [Online]. Available: https://downloads.regulations.gov/NHTSA-2016-0126-0387/attachment_2.pdf. [Accessed 22 November 2025].
- [14] B. Ahmed, A. W. Malik, T. Hafeez and N. Ahmed, "Services and simulation frameworks for vehicular cloud computing: a contemporary survey," *EURASIP Journal on Wireless Communications and Networking*, vol. 4, 2019.
- [15] S. Arif, S. Olariu, J. Wang, G. Yan, W. Yang and I. Khalil, "Datacenter at the Airport: Reasoning about Time-Dependent Parking Lot Occupancy," *IEEE Transactions on Parallel and Distributed Systems*, vol. 23, no. 11, pp. 2067-2080, 2012.
- [16] H. Goumidi, Z. Aliouat and S. Harous, "Vehicular Cloud Computing Security: A Survey," *Arabian Journal for Science and Engineering*, vol. 45, pp. 2473-2499, 2020.
- [17] A. Boukerche and R. E. De Grande, "Vehicular cloud computing: Architectures, applications, and mobility," *Computer Networks*, vol. 135, pp. 171-189, 2018.

- [18] A. Ashok, P. Steenkiste and F. Bai, "Vehicular Cloud Computing through Dynamic Computation Offloading," *Computer Communications*, vol. 120, pp. 125-137, 2018.
- [19] N. Keshari, D. Singh and A. K. Maurya, "A survey on Vehicular Fog Computing: Current state-of-the-art and future directions," *Vehicular Communications*, vol. 38, 2022.
- [20] X. Hou, Y. Li, M. Chen, D. Wu, D. Jin and S. Chen, "Vehicular Fog Computing: A Viewpoint of Vehicles as the Infrastructures," *IEEE Transactions on Vehicular Technology*, vol. 65, no. 6, pp. 3860-3873, 2016.
- [21] T. Ahmad, "Zero-Trust Mobility-Aware Authentication Framework for Secure Vehicular Fog Computing Networks," *arXiv:2506.05355*, 21 May 2025.
- [22] K. L. Lim, J. Whitehead, D. Jia and Z. Zheng, "State of data platforms for connected vehicles and infrastructures," *Communications in Transportation Research*, vol. 1, 2021.
- [23] IBM, "What is a message broker?," [Online]. Available: <https://www.ibm.com/think/topics/message-brokers>. [Accessed 22 November 2025].
- [24] P. Singh, "Middleware Support for Communication in Connected Vehicles," 9 August 2021. [Online]. Available: <https://publications.scss.tcd.ie/theses/diss/2021/TCD-SCSS-DISSERTATION-2021-064.pdf>. [Accessed 21 May 2025].
- [25] Z. Kegenbekov and A. Saparova, "Using the MQTT Protocol to Transmit Vehicle Telemetry Data," *Transportation Research Procedia*, vol. 61, pp. 410-417, 2022.
- [26] F. Pinzel, J. Holfeld, A. Olunczek, P. Balzer and O. Michler, "V2V- and V2X-Communication data within a distributed computing platform for adaptive radio channel modelling," in *2019 6th International Conference on Models and Technologies for Intelligent Transportation Systems (MT-ITS)*, Cracow, Poland, 2019.
- [27] A. Figueiredo, P. Rito, M. Luís and S. Sargento, "Mobility Sensing and V2X Communication for Emergency Services," *Mobile Networks and Applications*, vol. 28, pp. 1126-1141, 2022.
- [28] Y. Du, M. Chowdhury, M. Rahman, K. Dey, A. Apon, A. Luckow and L. B. Ngo, "A Distributed Message Delivery Infrastructure for Connected Vehicle Technology Applications," *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 3, pp. 787-801, 2017.
- [29] A. Hugo, B. Morin and K. Svantorp, "Briding MQTT and Kafka to support C-ITS: a feasibility study," in *21st IEEE International Conference on Mobile Data Management (MDM)*, Versailles, France, 2929.
- [30] HiveMQ, "Rimac Puts the Future of Connected Cars in the Fast Lane with HiveMQ," [Online]. Available: <https://www.hivemq.com/case-studies/rimac/>. [Accessed 22 November 2025].
- [31] Confluent, "Apache Kafka Native MQTT at Scale with Confluent Cloud and Waterstream," 15 July 2020. [Online]. Available: <https://www.confluent.io/blog/iot-streaming-use-cases-with-kafka-mqtt-confluent-and-waterstream>. [Accessed 22 November 2025].
- [32] A. Shvaika, D. Shvaika, D. Landiak and V. Artemchuk, "A distributed architecture for MQTT messaging: the case of TBMQ," *Journal of Big Data*, vol. 12, no. 224, 2025.
- [33] S. Shin, S. Eom and M. Choi, "Implementation of Edge Computing Platform for Smart Farm using MQTT and Kafka," *Journal of System and Management Sciences*, vol. 12, no. 1, pp. 158-174, 2022.
- [34] V. N. Gudivada, S. Ramaswamy and S. Srinivasan, "7 - Data Management Issues in Cyber-Physical Systems," in *Transportation Cyber-Physical Systems*, Cambridge, MA, Elsevier, 2018, pp. 173-200.
- [35] J. Fjällid, "A Comparative Study of Databases for Storing Sensor Data," 29 May 2019. [Online]. Available: <https://www.diva-portal.org/smash/get/diva2%253A1325707/FULLTEXT01.pdf>. [Accessed 22 November 2025].

- [36] L. M. M. Ferreira, F. Coetho and J. Pereira, "Databases in Edge and Fog Environments: A Survey," *ACM Computing Surveys*, vol. 56, no. 11, pp. 1-40, 2024.
- [37] D. Ku, H. Zang, A. Yusupov, S. Park and J. Kim, "Vehicle-to-Everything-Car Edge Cloud Managemnet with Development, Security, and Operations Automation Framework," *Electronics*, vol. 14, no. 3, 2025.
- [38] U. D. o. Transportation, "Connected Vehicle Pilot Deployment Program Phase 3 - Understanding and Enabling Cooperative Driving for Advanced Connected Vehicles in New York City," 29 December 2021. [Online]. Available: https://rosap.ntl.bts.gov/view/dot/63613/dot_63613_DS1.pdf. [Accessed 22 November 2025].
- [39] A. Rayamajhi, M. Rahman, M. Kaur, J. Liu, M. Chowdhury, H. Hu, J. McClendon, K.-C. Wang, A. Gosain and J. Martin, "ThinGs in a Fog: System Illustration with Connected Vehicles," in *85th IEEE Conference on Vehicular Technology (VTC)*, Sydney, 2017.
- [40] A. Flink, "What is Apache Flink? - Architecture," [Online]. Available: <https://flink.apache.org/what-is-flink/flink-architecture/>. [Accessed 22 November 2025].
- [41] A. Flink, "Use Cases," [Online]. Available: <https://flink.apache.org/what-is-flink/use-cases/>. [Accessed 22 November 2025].
- [42] T. Shwe and M. Aritsugi, "Optimizing Data Processing: A Comparative Study of Big Data Platforms in Edge, Fog, and Cloud Layers," *Applied Sciences*, vol. 14, no. 1, 2024.
- [43] C. Prehofer and S. Mehmood, "Big Data Architectures for Vehicle Data Analysis," in *2020 IEEE International Conference on Big Data*, Atlanta, Georgia, 2020.
- [44] B. Havers, R. Duvignau, H. Najdataei, V. Gulisano, M. Papatriantafilou and A. C. Koppisetty, "DRIVEN: A framework for efficient Data Retrival and clustering in Vehicular Networks," *Future Generation Computer Systems*, vol. 107, pp. 1-17, 2020.
- [45] J. O. Kephart and D. M. Chess, "The Vision of Autonomic Computing," *Computer*, vol. 36, no. 1, pp. 41-50, 2003.
- [46] D. M. Chess, C. C. Palmer and S. R. White, "Security in an autonomic computing environment," *IBM Systems Journal*, vol. 42, no. 1, pp. 107-118, 2003.
- [47] C. Rouff, L. Watkins, R. Sterritt and S. Hariri, "SoK: Autonomic Cybersecurity - Securing Future Disruptive Technologies," in *IEEE International Conference on Cyber Security and Resilience (CSR)*, Virtual, 2021.
- [48] C. Zhang, W. Shao, X. Wang, Y. Cao, A. H. J. Alhamadah, Y.-Z. Lin, P. Satam and L. Watkins, "Explainable Autonomic Cybersecurity System for Smart Power Grid," in *IEEE Conference on Communications and Network Security (CNS)*, Taipei, Taiwan, 2024.
- [49] G. Le Lann, "Autonomic Vehicular Networks: Safety, Privacy, Cybersecurity and Societal Issues," in *IEEE 87th Vehicular Technology Conference (VTC Spring)*, Porto, Portugal, 2018.
- [50] T. K. Venkatasamy, J. Hossen, G. Ramasamy and N. H. B. A. Aziz, "Intrusion detection system for V2X communication in VANET networks using machine learning-based cyrtographic protocols," *Nature, Scientific Reports*, 30 December 2024.
- [51] S. T. Banafshehvaragh, M. Zarei and A. M. Rahmani, "A reliable score-based routing protocol using a fog-assisted intrusion detection system in vehicular ad-hoc networks," *Nature, Scientific Reports*, 16 July 2025.
- [52] R. Sedar, C. Kalalas, F. Vazquez-Gallego, L. Alonso and J. Alonso-Zarate, "A Comprehensive Survey of V2X Cybersecurity Mechanisms and Future Research Paths," *IEEE Open Journal of the Communications Society*, vol. 4, pp. 325-391, 2023.
- [53] J. M. Tines, M. Aldeen, A. Enan, M. S. Salek, L. Cheng and M. Chowdhury, "Real-World Evaluation of Protocol-Compliant Denial-of-Service Attacks on C-V2X-based Forward Collision Warning Systems," *Cryptography and Security. arXiv:2508.02805*, 4 August 2025.
- [54] M. Islam, M. Chowdhury, H. Li and H. Hu, "Cybersecurity attacks in vehicle-to-infrastructure applications and their prevention," *Cryptography and Security. arXiv:1711.10651*, 29 November 2017.

- [55] V. H. Le, J. den Hartog and N. Zannone, "Feature Selection for Anomaly Detection in Vehicular Ad Hoc Networks," in *15th International Joint Conference on e-Business and Telecommunications*, Porto, Portugal, 2018.
- [56] A. Lekidis, "Anomaly detection mechanisms for in-vehicle and V2X systems," in *ARES '24: Proceedings of the 19th International Conference on Availability, Reliability and Security*, New York, NY, 2024.
- [57] wolfMQTT, "MQTT v.50 versus v3.1.1," wolfSSL, 17 May 2024. [Online]. Available: <https://www.wolfssl.com/mqtt-v5-0-versus-v3-1-1/>. [Accessed 11 December 2025].
- [58] Confluent, "confluent-kafka - PyPI," Confluent, 7 November 2025. [Online]. Available: <https://pypi.org/project/confluent-kafka/>. [Accessed 1 December 2025].
- [59] S. Bradshaw, E. Brazil and K. Chodorow, *MongoDB: The Definitive Guide*, 3rd Edition, Sebastapol, CA: O'Reilly Media, Inc., 2019.
- [60] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper and T. Neumann, "How Good Are Query Optimizers, Really?," *Proceedings of the VLDB Endowment*, vol. 9, no. 3, pp. 204-215, 2015.
- [61] M. Rigger and Z. Su, "Testing Database Engines via Pivoted Query Synthesis," *14th USENIX Symposium on Operating Systems Design and Implementation*, pp. 667-682, 2020.
- [62] P. Carbone, S. Ewen and S. Haridi, "Apache Flink: Stream and Batch Processing in a Single Engine," *The Bulletin of the Technical Committee on Data Engineering*, vol. 38, no. 4, 2015.
- [63] F. Hueske and V. Kalavri, *Stream Processing with Apache Flink*, Sebastapol, CA: O'Reilly Media, Inc., 2019.
- [64] C. 2. C. C. Consortium, "White Paper on Misbehaviour Detection and Reporting to Misbehaviour Authority," 17 December 2021. [Online]. Available: https://www.car-2-car.org/fileadmin/documents/General_Documents/C2CCC_WP_2092_MisbehaviourDetection_and_Reporting_V1.0.pdf. [Accessed 23 November 2025].
- [65] Y. Byzek, "Optimizing Your Apache Kafka Deployment," Confluent, 2020.
- [66] F. Goncalves, F. Ribeiro, O. Gama, J. Santos, A. Costa, B. Dias, M. J. Nicolau, J. Macedo and A. Santos, *V2X Security Threats - Dataset*, 2020.
- [67] F. Goncalves, B. Ribeiro, O. Gama, J. Santos, A. Costa and B. Dias, "Synthesizing Datasets with Security Threats for Vehicular Ad-Hoc Networks," in *IEEE Conference on Global Communications (GLOBECOM)*, Taipei, Taiwan, 2020.
- [68] M. S. Rama and G. Padmavathi, "Self-healing AIS with Entropy Based SVM and Bayesian Aggregate Model for the Prediction and Isolation of Malicious Nodes Triggering DoS Attacks in VANET," *International Journal for Computer Network and Information Security*, vol. 3, pp. 90-105, 223.
- [69] D. Hamilton, K. Kornegay and L. Watkins, "Autonomous navigation assurance with explainable AI and security monitoring," in *IEEE Applied Imagery Pattern Recognition Workshop (AIPR)*, Washington, D.C., 2020.

9. Appendix

9.1: V2X Communication Metrics

V2X Phase	Message Types	Urban	Suburban	Rural/Highway
Awareness Driving	Basic Safety Message (BSM)	3	6	10
	Signal Phase and Time (SPaT) MAP Message (MAP) In-Vehicle-Information Message (IVI)	10	25	50
	Personal Safety Message (PSM)	1	1	10
Sensing Driving	Collective Perceptive Message (CPM)	3	6	10
Cooperative Driving	Manuever Coordination Message (MCM)	3	6	10
	Platooning Control Message (PCM)	50	50	50
TOTAL (Hz)		70	94	140

Figure A.1: Expected number of messages per second for each type based on C2C-CC estimates.

V2X Phase	Message Types	Urban	Suburban	Rural/Highway
Awareness Driving	Basic Safety Message (BSM)	380	380	380
	Signal Phase and Time (SPaT) MAP Message (MAP) In-Vehicle-Information Message (IVI)	1200	1200	1200
	Personal Safety Message (PSM)	350	350	350
Sensing Driving	Collective Perceptive Message (CPM)	1000	1000	1000
Cooperative Driving	Manuever Coordination Message (MCM)	1000	1000	1000
	Platooning Control Message (PCM)	400	400	400
TOTAL (Hz)		4330	4330	4330

Figure A.2: Expected bytes per second usage for each message type based on C2C-CC estimates.

V2X Phase	Message Types	Urban	Suburban	Rural/Highway
Awareness Driving	Basic Safety Message (BSM)	1140	2280	3800
	Signal Phase and Time (SPaT) MAP Message (MAP) In-Vehicle-Information Message (IVI)	12000	30000	60000
	Personal Safety Message (PSM)	350	350	3500
Sensing Driving	Collective Perceptive Message (CPM)	3000	6000	10000
Cooperative Driving	Manuever Coordination Message (MCM)	3000	6000	10000
	Platooning Control Message (PCM)	20000	20000	20000
TOTAL (Hz)		39490	64630	107300

Figure A.3: Expected total bytes per second usage for each message type.

9.2: Hopkins V2X Testbed

9.2.1: Testbed Integration and Data Flow

The smartphone acts as a rich sensor node and serves as the internet interface for the Raspberry Pi, forming the core of the onboard data acquisition subsystem. The custom SensaGram app on the phone streams sensor readings over UDP to the Sensor Dump application running on the Raspberry Pi. This application preprocesses the data and securely transmits it to a central server using the campus VPN (WireGuard), ensuring privacy and consistent connectivity regardless of the network environment.

Once received at the data platform, the Sensor Dump server module stores the sensor data in a MongoDB NoSQL database, using BSON format for time-stamped series. Video data, streamed using the IP Camera app, traverses the same routing path and is made accessible for live visualization via MediaMTX, although persistent recording is not currently supported due to bandwidth limits.

9.2.2: System Architecture Context

This architecture facilitates loose coupling between simulation and platform layers. The hardware/software stack—consisting of the Raspberry Pi (ARM Debian), Android phone, and a Red Hat Linux server—ensures that experimental code and analytics workflows can access real testbed data or simulation outputs in a consistent format. The testbed's TCP server-client architecture supports future extensions for direct bidirectional communication, enabling dynamic experiment control and real-time adaptive feedback loops.

9.2.3: Data Usage in the Testbed

The collected sensor data serves multiple research needs:

- Real-time monitoring and logging of vehicle telemetry (location, orientation, velocity, and environmental context)
- Validation of cooperative V2X algorithms, including Basic Safety Messages (BSMs) and other V2X message types
- Application to machine learning pipelines for security/event detection (such as DoS detection described in section 3.3)
- Live perception/awareness use cases, such as driver/observer feedback, safety auditing, or incident response simulation